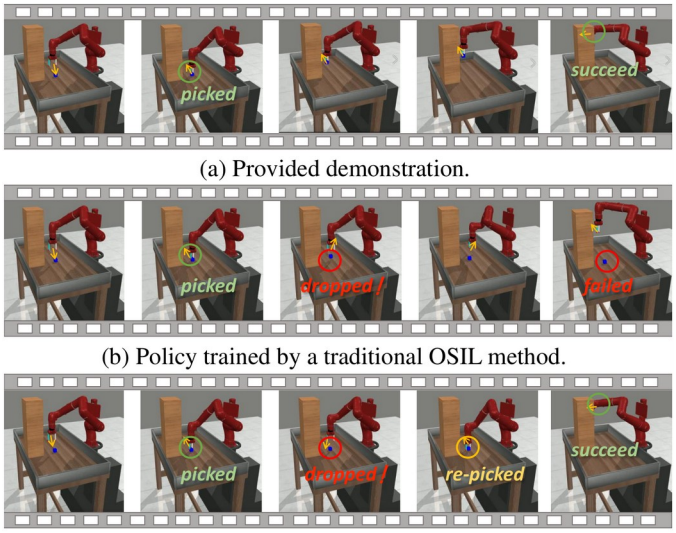
DDT: Teaching an Imitator to Trace, Not Replay, a Single Demonstration

ICML 2024 - Nanjing University and Polixir Technologies on one-shot imitation that survives unforeseen changes

One-shot imitation learning (OSIL) asks an agent to carry out a task after watching a single demonstration. It works remarkably well when the deployment environment looks like the one where the demonstration was recorded. The trouble is that the real world rarely holds still. After a route is demonstrated, a truck can park across the path; after a grasp is shown, the object can slip out of the gripper. These are exactly the moments where a demonstration stops being a script to replay and becomes, at best, a rough guide.

Most OSIL methods embed the demonstration as a context vector and clone the demonstrated actions. In a stationary world that is enough. But once an unforeseen change pushes the agent into a state the demonstration never covered, blind replay has nothing sensible to say - and existing methods degrade sharply. This paper, presented at ICML 2024, takes that failure mode as its subject.

The authors' answer is Deep Demonstration Tracing (DDT): instead of replaying a demonstration, the agent learns to trace it. It figures out which demonstrated states are relevant right now, reads off how the expert behaved there, and steers back onto the demonstrated path after a detour. DDT pairs a purpose-built demonstration transformer with a meta-reinforcement-learning training scheme, and it holds up where prior one-shot imitation methods collapse.



(a) Provided demonstration.

(b) Policy trained by a traditional OSIL method.

(c) Policy trained by DDT.

Figure 1. A pick-and-place task in Meta-World where the object slips mid-execution. Given the demonstration (a), a traditional OSIL policy (b) drops the object and fails, while the DDT policy (c) re-picks the object and still succeeds - the difference between replaying actions and tracing intent.

Paper: https://arxiv.org/abs/2408.05285 | Code: https://github.com/xionghuichen/Deep-Demonstration-Tracing | Project page: https://osil-ddt.github.io

When replay is not enough

Formally, a task is a Markov decision process (MDP), and OSIL provides one expert trajectory as the only description of what the agent should do. Prior work has largely studied the setting where collection and deployment are near-identical, so a policy that reproduces the demonstrated action at each demonstrated state does fine. The paper deliberately widens the gap between collection and deployment - adding obstacles, disturbances, and other unforeseen changes at runtime - and shows that this is where the interesting difficulty lives, and where it has been under-studied.

The requirement, then, is not merely to match demonstrated actions but to behave sensibly off the demonstrated path and to find the way back. That is an adaptivity current methods do not build in.

A blueprint borrowed from human imitation

The design starts from how a person imitates a route. Following a demonstrated path toward a destination, you meet a truck parked where the demonstration had none. You do not freeze or restart; you detour around the obstacle and rejoin the original path at a convenient point. The authors distill this into a three-stage decision process: identify which demonstrated states are relevant to the current situation, analyze how the expert behaved in those states, and trace back onto the demonstration.

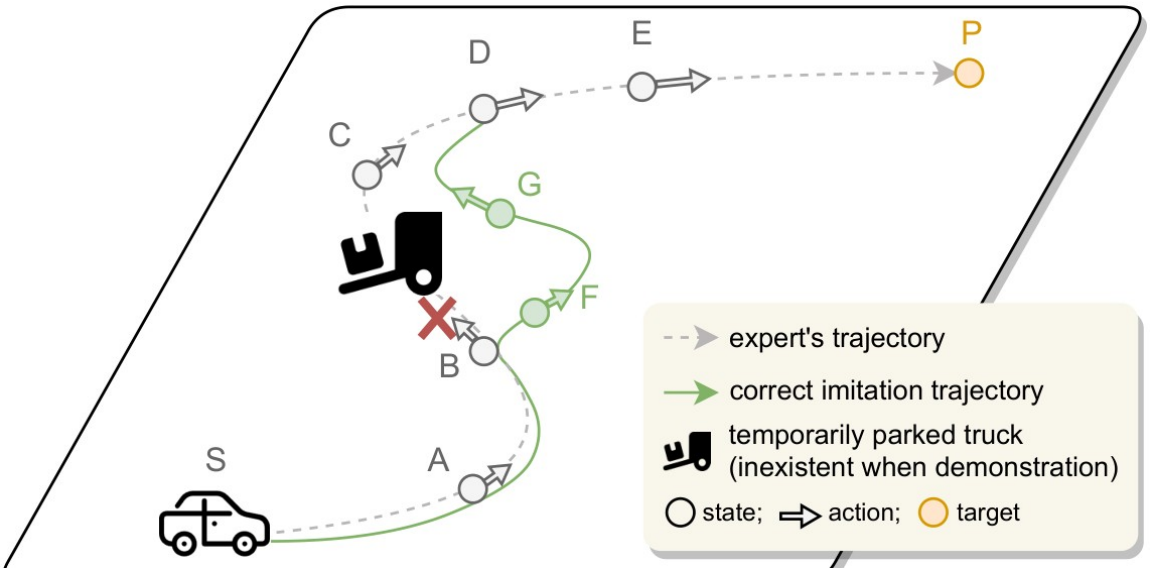


Figure 2. The human blueprint for one-shot imitation under unforeseen change. The expert trajectory (dashed) runs from start S to target P; a truck (absent when the demonstration was collected) blocks state B. A capable imitator leaves the demonstrated path, detours (green), and rejoins the demonstration - identify, analyze, then trace back.

This blueprint is not just narrative color: it becomes the literal structure of the network. Each of the three stages maps onto a specific computation in the demonstration transformer, which is what turns an intuition about human behavior into an inductive bias the model can learn with.

The demonstration transformer

The core module treats the agent's current state as a query and the demonstration's state-action pairs as keys and values. An attention-weighting module scores which demonstrated states matter now (identify). A point-wise multiplication then combines those weights with the encoded expert behavior to read off what the expert did there (analyze). The result is folded back together with the current state and passed through an MLP to produce the action (trace). The expert-state encoder and the visited-state encoder share weights, so demonstrated and current states are compared in the same representation space.

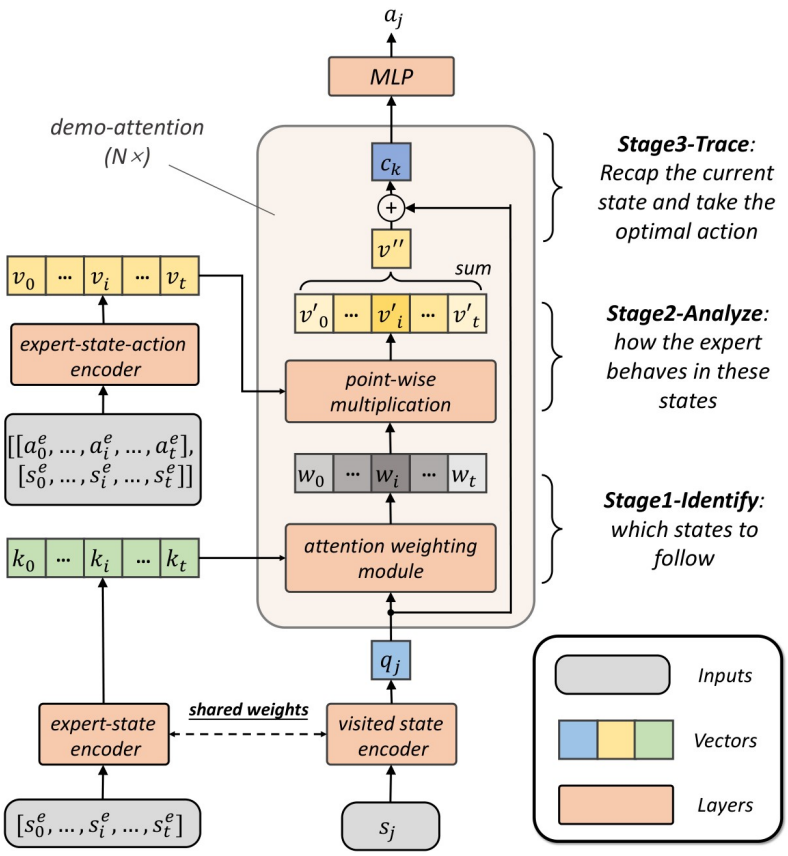


Figure 3. The demonstration transformer. Reading bottom to top, the three human stages become network operations: attention weighting (Stage 1, identify which states to follow), point-wise multiplication (Stage 2, analyze how the expert behaved), and recombination with the current state through an MLP (Stage 3, trace back and act). The expert-state and visited-state encoders share weights.

Rolling the demonstration encoder and the context-based policy into a single architecture is what gives DDT its inductive bias toward tracing. That bias is also what lets it generalize to demonstrations - and to disturbances - it never saw during training, rather than memorizing a fixed mapping from context to action.

Training it as meta-RL, not behavior cloning

The second ingredient is how the policy is trained. Behavior cloning can only teach the agent what to do at states the demonstration visited, which is precisely the wrong tool when the

whole problem is behaving well off the demonstrated path. DDT instead casts one-shot imitation as context-based meta-reinforcement learning: the demonstration is the context, and the agent learns by interaction to act even in states the demonstration never reached.

The reward is a stationary OSIL reward that combines a term rewarding the agent for following the demonstration with a large-weighted task reward that keeps the focus on actually completing the task. This dense signal is what makes learning efficient early on, and the whole objective is optimized with Soft Actor-Critic (SAC). The paper also gives a theoretical condition on the environment - roughly, that it is recoverable back onto the demonstration - under which one trajectory suffices for imitation.

VPAM: a benchmark built for unforeseen change

To test imitation under runtime change, the authors build Valet Parking Assist in Maze (VPAM), inspired by real-world valet parking. A point agent must travel from a start to a target in a maze it has never seen globally, relying only on short local views computed from eight rays over five steps. Crucially, rectangular obstacles are randomly placed on the route and often did not exist in the demonstration, so the agent cannot simply replay demonstrated actions. Eight task variants toggle whether it is a single map or many, whether obstacles appear, and whether the agent is told its own coordinates.

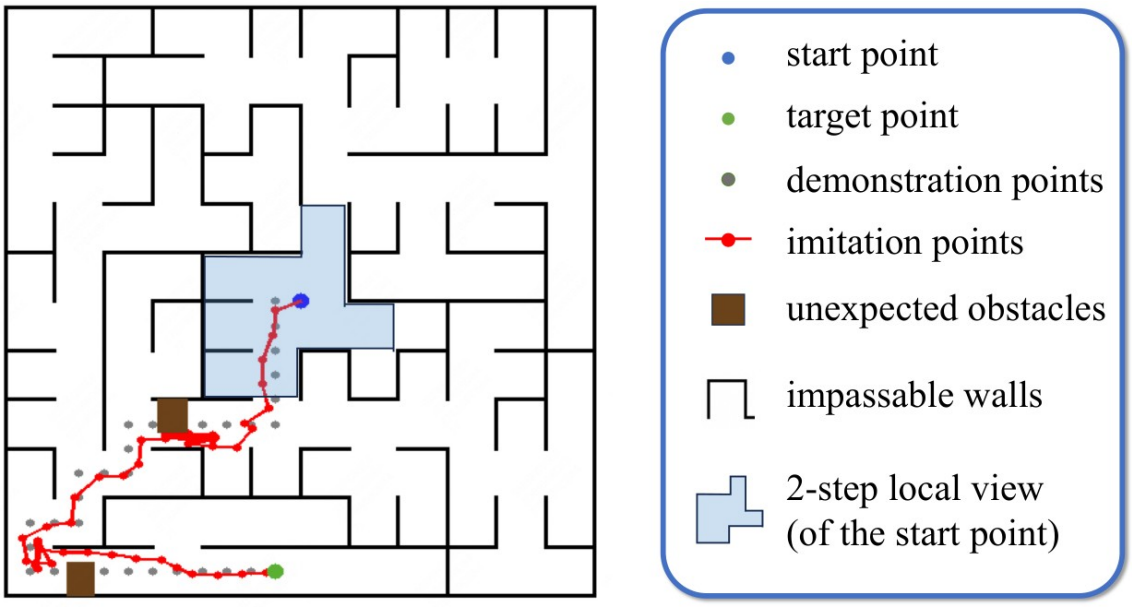


Figure 4. A VPAM maze. The agent moves from the start (blue) to the target (green) guided by the demonstration points (gray), while unexpected obstacles (brown) block the demonstrated route. The agent perceives only a small local view (shaded), and DDT's imitation trajectory (red) detours around each obstacle and rejoins the demonstration.

Beyond VPAM, DDT is also evaluated on robotics benchmarks: Meta-World with added disturbances, Gymnasium's Reacher and Pusher, and a MuJoCo manipulation task involving grasping, stacking, and clutter, along with partially observable variants of the VPAM maze itself.

How well does it hold up?

On VPAM, DDT leads across every setting - and, more tellingly, it degrades far less as conditions get harder. Moving from the training distribution to non-obstacle test maps to unforeseen obstacles, DDT drops only from 0.86 to 0.84 to 0.73, a 15% fall. The baselines fall 20%, 33%, and 52% over the same span, so DDT retains at least 5% more of its performance than any competitor.

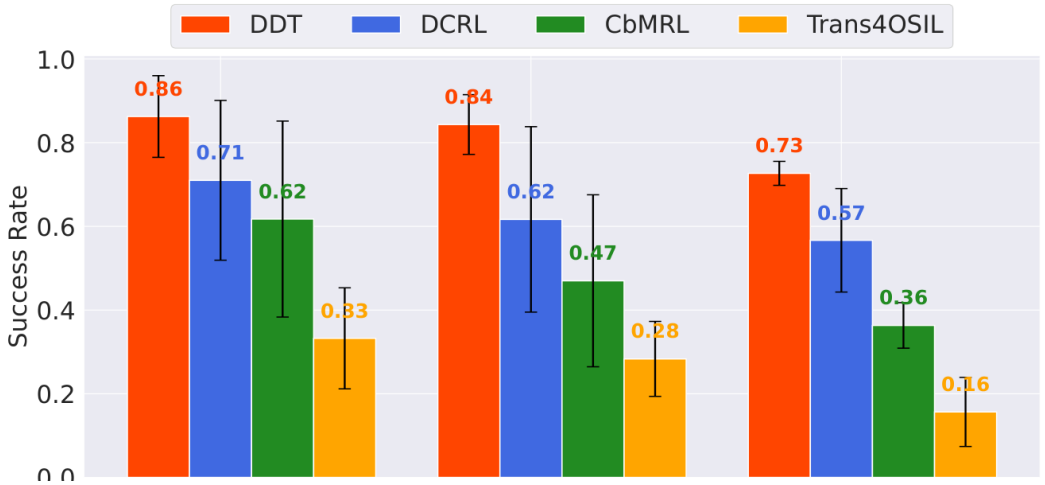


Figure 5. Success rates on VPAM across three settings. DDT (red) leads DCRL, CbMRL, and the behavior-cloning Trans4OSIL in every setting, and the gap widens under unforeseen obstacles - where DDT reaches 0.73 against the best baseline's 0.57. Black bars show standard error across tasks over three seeds.

Table 1. VPAM success rates (higher is better) and degradation from Train to the Unforeseen-Obstacle setting.

Method	Train	Non-Obstacle	Unforeseen Obstacle	Degradation
DDT (ours)	0.86	0.84	0.73	-15%
DCRL	0.71	0.62	0.57	-20%
CbMRL	0.62	0.47	0.36	-33%
Trans4OSIL	0.33	0.28	0.16	-52%

The robotics results tell the same story more starkly. On Meta-World with disturbance, DDT succeeds 61% of the time on unseen demonstrations, whereas the strongest baseline manages at most about 12%. The meta-RL mechanism, not just the transformer, is what buys this robustness to change.

Is it really tracing?

A method named for tracing should be shown to trace. The authors visualize a DDT rollout in a maze together with the attention scores the demonstration transformer places over the expert trajectory as the agent moves. If DDT were following the demonstration, the attention should shift along the expert path roughly in step with the agent's own progress.

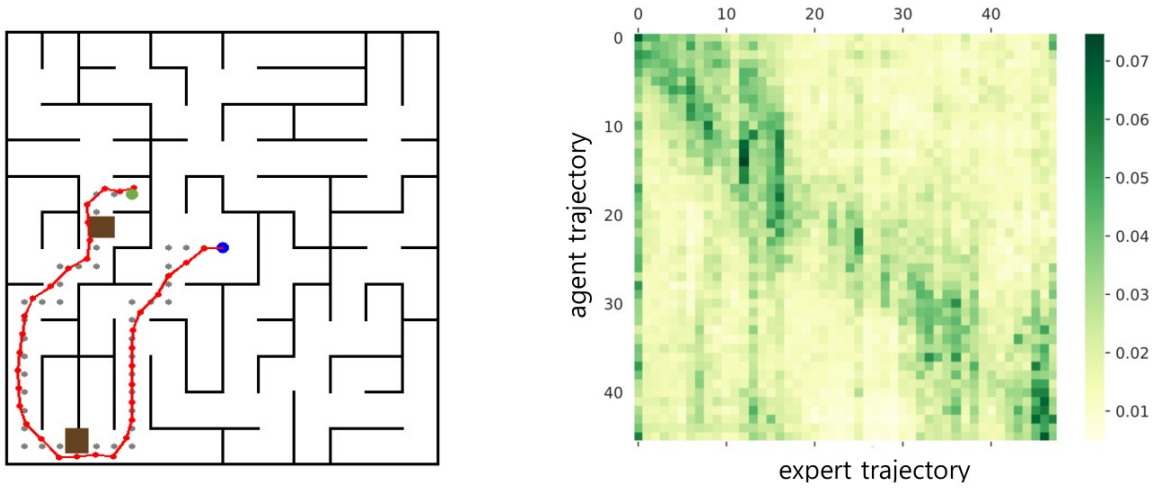


Figure 7. Left: a trajectory generated by DDT, detouring around obstacles and returning to the demonstrated route. Right: the attention score map, agent step (vertical) against expert step (horizontal). The bright band runs down the diagonal - as the agent advances, its attention tracks the corresponding part of the demonstration, which is tracing made visible.

That diagonal is the mechanism working as intended: attention follows the demonstration in step with the agent, breaking away exactly when a detour is needed and re-locking afterward. It is a satisfying check that the architecture is doing the thing its design was meant to encourage.

Ablations and scaling

Two ablations confirm that both ingredients matter. Swapping the demonstration transformer for a standard transformer noticeably lowers asymptotic performance - so it is the tailored architecture, not attention in general, that drives the imitation ability. Removing the OSIL reward and learning only from the sparse ending reward sharply slows learning, because the OSIL reward supplies the dense signal that lets the agent follow the demonstration early in training.

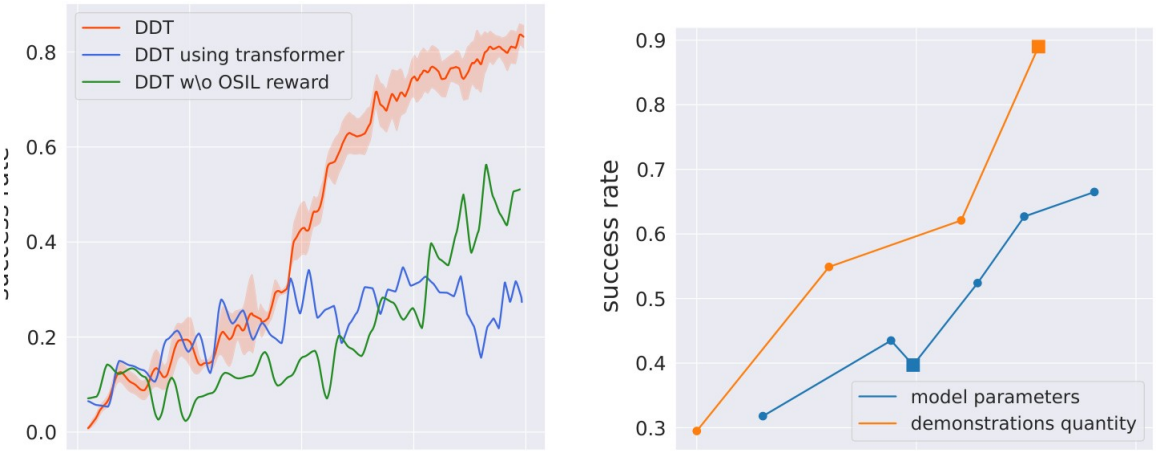


Figure 8. Left: learning curves (success rate vs. environment steps) for DDT and its ablations; replacing the demonstration transformer or dropping the OSIL reward both hurt. Right: asymptotic success rate as data and model size scale up (log x-axis) - performance climbs log-linearly, with roughly a 2x gain from scaling model parameters.

The scaling curves are a quieter but intriguing result: DDT improves log-linearly with both data volume and model size, roughly doubling as parameters grow. That clean scaling behavior is what leads the authors to suggest DDT could serve as a backbone for larger, more general decision-making agents.

Takeaway

The core idea is small and durable: an agent handed a single demonstration should learn to trace it, not replay it - identifying which demonstrated states are relevant now, reading off what the expert did, and steering back onto the path after a detour. DDT operationalizes this with a demonstration transformer trained by meta-RL, and the payoff is one-shot imitation that stays robust when the environment changes at runtime, a regime where earlier methods fail.

The honest boundaries are worth keeping in view: the single-demonstration guarantee rests on a recoverability condition, and the headline results live on a purpose-built maze benchmark alongside a set of robotics tasks. Within that scope, though, DDT is a clean demonstration that the right inductive bias plus reinforcement learning beats behavior cloning when the world refuses to stay still - and its scaling behavior hints at more to come.